

CHAPTER 05

05. Express 중급 — 커지는 앱을 정리하는 세 가지 도구

Node.js · Express · NestJS 강의 · 2026

04장에서 게시물 API는 한 파일에 다 들어 있었다. 라우트도, 데이터도, 본문 검사도, "못 찾았다"는 응답도 모두 같은 파일에 섞여 있었다. 예제 하나로도 그래도 된다. 그런데 자원이 게시물·댓글·사용자로 늘고, 검사할 항목이 많아지고, 에러를 돌려주는 코드가 라우트마다 반복되기 시작하면 한 파일은 금세 버틸 수 없다.

이 장은 그 "흠어진 것을 모으는" 작업이다. 04장에서 라우트마다 따로 적던 404 응답을 한곳으로 모으고(에러 처리), 한 파일에 뭉쳐 있던 라우트를 자원별 파일로 가르고(라우터 분리), 핸들러 첫머리에 손으로 적던 `if (!title)` 검사를 라이브러리에 맡긴다(입력 검증). 새 기능을 배우는 장이라기보다, **앱이 커져도 무너지지 않게 구조를 잡는** 장이다.

이 자료는 **개념 예습용**이다. 실제 수업에서 쓰는 파일 이름과 폴더 구성, 다루는 범위는 강사에 따라 다를 수 있다. 그래서 여기서는 파일이 아니라 **무엇을 배우는가**에 초점을 둔다.

이 장에서 배우는 것

- 에러를 한곳으로 모으는 법: 라우트에서 `throw` 하고, **인자 4개짜리 에러 핸들러**가 받는다
- 없는 경로(404)를 마지막 미들웨어 하나로 처리하는 법
- `express.Router()` 로 라우트를 **자원별 파일**로 나누고 메인 앱 파일에서 마운트하는 법
- `Joi` 스키마로 입력 규칙을 **선언적으로** 정의하고, 고차함수로 검증 미들웨어를 짚어내는 법

내용은 세 주제로 나뉘고, 각각 독립적으로 동작하는 작은 서버다.

주제	핵심
에러 처리	404 핸들러 + 중앙 에러 핸들러(인자 4개)
라우터 분리	<code>express.Router()</code> 로 자원별 파일 분리
입력 검증	Joi 스키마로 입력 검증

세 주제 중 "라우터 분리"는 한 파일이 아니라 여러 파일이 역할을 나눠 맡는 구조다. 자원마다 라우트를 따로 모은 **자원별 라우터 파일**들을, **메인 앱 파일**이 가져와 경로에 붙이는 모양이다.

```

메인 앱 파일      # 라우터를 가져와 경로에 마운트하고 서버를 띄운다
└─ 자원별 라우터 파일
   └─ 게시물 라우터    # /posts 관련 라우트
   └─ 댓글 라우터      # /comments 관련 라우트
  
```

핵심 개념 — 반복되는 것은 한곳으로 모은다

04장에서 "못 찾으면 404"를 라우트마다 적었다. 단건 조회에도, 수정에도, 삭제에도 똑같은 모양의 줄이 들어갔다.

```
if (!post) {
  return res.status(404).json({ message: "게시물을 찾을 수 없습니다." });
}
```

라우트가 셋이면 세 번, 자원이 셋이면 아홉 번이다. 같은 코드가 여기저기 흩어져 있으면, 메시지 형식 하나 바꾸는 데도 전부 찾아 고쳐야 한다. 중급의 핵심은 이 반복을 **한곳으로 모으는** 것이다.

세 가지 도구가 각자 다른 반복을 거둬들인다.

- **에러 처리** — 라우트마다 적던 실패 응답을, 앱 끝의 핸들러 하나로 모은다. 라우트는 "문제가 생겼다"고 `throw` 만 하고, 응답을 어떻게 돌려줄지는 한곳에서 결정한다.
- **라우터 분리** — 한 파일에 뭉친 라우트를, 자원별 파일로 가른다. 게시물 일은 게시물 라우터 파일에, 댓글 일은 댓글 라우터 파일에. 메인 앱 파일은 이들을 모아 붙이기만 한다.
- **입력 검증** — 핸들러 첫머리의 손검사를, 스키마라는 **규칙 선언**으로 바꾸고 미들웨어로 떼어낸다. 라우트는 "검증을 통과한 본문"만 받는다.

관통하는 한 가지 — 세 도구는 결국 **미들웨어**라는 한 개념의 변주다. 04장에서 미들웨어는 "요청 파이프라인의 한 칸"이고 `next()` 로 다음 칸에 넘긴다고 배웠다. 에러 핸들러는 인자가 4개인 미들웨어, 라우터는 묶음으로 마운트하는 미들웨어, 검증기는 라우트 앞에 끼우는 미들웨어다. 새 개념이 셋 늘어난 게 아니라, 아는 개념 하나를 세 자리에 쓰는 것이다.

준비

예제 폴더에서 `npm install` 로 의존성을 설치한다. 04장은 `express` 하나였는데, 이 장은 입력 검증을 위해 `joi` 가 하나 더 붙는다.

```
npm install      # express, joi 두 개가 설치된다
```

설치가 끝나면 서버 파일을 실행한다. 서버가 뜨면 다른 터미널을 열어 `curl` 로 요청을 보낸다. 한 번에 하나씩 실행하고, 종료는 `Ctrl+C` 다.

에러 처리 — `throw` 와 인자 4개 에러 핸들러

04장에서는 실패할 때마다 라우트 안에서 직접 `res.status(.).json(.)` 을 돌려줬다. 그게 틀린 건 아니지만, 같은 패턴이 자주 반복된다. 여기서는 그 반복을 **앱 끝의 핸들러 하나로** 거둬들인다. 라우트는 문제가 생기면 `throw` 만 하고, 응답을 만드는 일은 한곳에 맡긴다.

```
const express = require("express");
const app = express();

app.use(express.json());

const posts = [{ id: 1, title: "첫 번째 글", author: "김철수" }];
```

여기까지는 04장과 같다. `express.json()` 도 그대로 미들웨어다. 달라지는 건 라우트 안쪽과, 그 뒤에 새로 붙는 두 미들웨어다.

라우트에서 에러를 던진다 — `throw`

```
app.get("/posts/:id", (req, res) => {
  const id = Number(req.params.id);

  // 숫자가 아니면 에러를 던진다 - Express는 라우트에서 throw된 에러를
  // 자동으로 아래 "인자 4개" 에러 핸들러로 보낸다.
  if (Number.isNaN(id)) {
    const err = new Error("id는 숫자여야 합니다.");
    err.status = 400; // 이 값이 응답 상태코드가 된다.
    throw err;
  }

  const post = posts.find((p) => p.id === id);
  if (!post) {
    return res.status(404).json({ message: "게시물을 찾을 수 없습니다." });
  }
  res.json(post);
});
```

`/posts/abc` 처럼 숫자가 아닌 `id` 가 들어오면 `Number("abc")` 는 `NaN` 이다. 04장이라면 여기서 바로 `res.status(400).json(.)` 을 돌려줬겠지만, 이번엔 에러 객체를 만들어 `throw` 한다.

- `const err = new Error("id는 숫자여야 합니다.");` — 에러 객체를 만든다. 메시지는 나중에 응답 본문이 된다.
- `err.status = 400` — 에러 객체에 `status` 속성을 직접 붙인다. 표준 속성은 아니고, 우리가 약속한 값이다. 이 값을 뒤의 에러 핸들러가 읽어 응답 상태코드로 쓴다.
- `throw err` — 에러를 던진다. 여기서 핸들러 실행이 끊긴다.

핵심은 던진 에러가 그냥 사라지지 않는다는 점이다. Express는 라우트 핸들러에서 `throw` 된 에러를 받아, 앱에 등록된 에러 전용 미들웨어로 자동으로 넘긴다. 그래서 라우트는 응답을 만들 필요 없이 "문제가 생겼다"고 던지기만 하면 된다.

throw 나 res 나 — 같은 파일 안에서 두 방식이 섞여 있다. 잘못된 `id` 는 `throw` 로 보내고, 없는 글(`!post`) 은 `res.status(404)` 로 직접 돌려준다. 일부러 둘을 나란히 뒤서 비교용으로 보여준다. 실무에서는 보통 한쪽으

로 통일한다 — 실패를 전부 `throw` 로 보내 에러 핸들러 한곳에서 처리하면, 라우트는 정상 흐름만 남아 읽기 쉬워진다.

없는 경로를 받는다 — 404 미들웨어

```
// 404 - 위에서 어떤 라우트에도 안 걸린 요청이 여기로 온다.
app.use((req, res) => {
  res.status(404).json({ message: `${req.originalUrl} 경로가 없습니다.` });
});
```

`/unknown` 처럼 등록한 적 없는 경로로 요청이 오면, 위의 어떤 `app.get` 에도 걸리지 않는다. Express는 등록된 순서대로 위에서 아래로 라우트를 훑는데, **끝까지 아무도 안 받으면** 이 `app.use` 에 닿는다. 그래서 이 미들웨어는 반드시 모든 라우트 뒤에 뒤야 한다. 위에 두면 모든 요청을 가로채 404를 돌려준다.

- 이 미들웨어는 인자가 `(req, res)` 둘이다. `next` 가 없으니 라우트 핸들러처럼 보이지만, 경로를 지정하지 않은 `app.use` 라 남은 모든 요청의 종착지 역할을 한다.
- `req.originalUrl` — 요청이 들어온 원래 경로 전체다. `/unknown` 을 그대로 메시지에 담아 어떤 경로가 없었는지 알려준다.

에러를 모아 응답한다 — 인자 4개 에러 핸들러

```
// 중앙 에러 핸들러 - 인자가 4개(err, req, res, next)면 에러 전용 미들웨어다.
// 앱 어디서든 throw 하거나 next(err) 한 에러가 전부 여기로 모인다.
app.use((err, req, res, next) => {
  console.error(err.message);
  res.status(err.status || 500).json({ message: err.message });
});
```

이 장에서 가장 중요한 다섯 줄이다. 04장의 미들웨어는 인자가 `(req, res, next)` 셋이었다. 여기는 **앞에 `err` 이 하나 더 붙어 넷이다**. Express는 이 인자 개수만 보고 "이건 에러 전용 미들웨어구나" 하고 구분한다. 넷이면 에러 핸들러, 셋이면 일반 미들웨어다. **인자가 넷이라는 사실 자체가 약속이라**, `next` 를 안 써도 자리를 지워선 안 된다.

- 앱 어디서든 `throw` 된 에러, 그리고 `next(err)` 로 넘긴 에러가 전부 이 핸들러 하나로 모인다. 흩어졌던 실패 처리가 한곳에 모이는 지점이다.
- `res.status(err.status || 500)` — 앞서 라우트에서 붙인 `err.status` (여기선 400)를 읽어 상태코드로 쓴다. `status` 가 없는 에러라면 500 (서버 내부 오류)을 기본으로 쓴다. 예상한 에러는 400-404 같은 적절한 코드로, 예상 못 한 에러는 500으로 나간다.
- `console.error(err.message)` — 서버 로그에 에러를 남긴다. 클라이언트에 응답을 돌려주는 것과 별개로, 무슨 일이 있었는지 서버 쪽에도 기록을 남긴다.

요청 하나가 에러로 끝나는 흐름은 이렇다.

GET /posts/abc → 라우트 핸들러 — throw err(status 400) → 에러 핸들러(err, req, res, next) → 400 응답

미들웨어	인자	언제 실행되나
일반 미들웨어	(req, res, next)	모든 요청이 라우트 전에 거친다
404 핸들러	(req, res)	어떤 라우트에도 안 걸린 요청
에러 핸들러	(err, req, res, next)	throw 또는 next(err) 로 에러가 넘어올 때

순서가 전부다 — 이 세 줄은 반드시 **라우트 → 404 → 에러 핸들러** 순으로 뒤야 한다. 라우트가 먼저 요청을 받아보고, 아무도 못 받은 경로는 404가 잡고, 던져진 에러는 맨 끝 에러 핸들러가 받는다. 에러 핸들러를 위로 올리면 에러가 도달하지 못한다.

라우터 분리 — express.Router() 로 자원별 파일 나누기

04장에서는 게시물 라우트가 전부 한 파일에 있었다. 게시물만 있을 땐 괜찮다. 그런데 댓글이 생기고 사용자가 생기면, 한 파일에 `app.get("/posts")`, `app.get("/comments")`, `app.get("/users")` 가 수십 개씩 쌓인다. 어디까지가 게시물 코드이고 어디부터가 댓글 코드인지 눈으로 가르기 어려워진다.

`express.Router()` 는 라우트 묶음을 **작은 앱처럼** 따로 만들어, 자원별 파일로 떼어낸다. 게시물 라우트는 게시물 라우터 파일에, 댓글 라우트는 댓글 라우터 파일에 두고, 메인 앱 파일은 이 묶음들을 가져와 경로에 붙이기만 한다.

자원별 라우터 파일 — 한 자원의 라우트만 모은다

```
// 게시물 라우터 - 메인 앱 파일에서 "/posts" 경로에 마운트된다.
// 그래서 여기서 "/"는 실제로 "/posts", "/:id"는 "/posts/:id"가 된다.
const express = require("express");
const router = express.Router();

const posts = [
  { id: 1, title: "첫 번째 글", author: "김철수" },
  { id: 2, title: "두 번째 글", author: "이영희" },
];

router.get("/", (req, res) => {
  res.json(posts);
});

router.get("/:id", (req, res) => {
  const post = posts.find((p) => p.id === Number(req.params.id));
  if (!post) {
    return res.status(404).json({ message: "게시물을 찾을 수 없습니다." });
  }
  res.json(post);
});

module.exports = router;
```

app 자리에 router 가 들어왔을 뿐, 라우트를 등록하는 모양은 04장과 똑같다. router.get(.), router.post(.) 처럼 쓴다. 차이는 두 가지다.

- `const router = express.Router()` — `express()` 대신 `express.Router()` 를 부른다. 서버를 띄우는 `app` 이 아니라, 라우트만 담는 작은 묶음을 만든다. `listen` 도 `express.json()` 도 여기엔 없다. 오직 게시물 라우트만 모은다.
- 경로가 짧아졌다 — `router.get("/")` 와 `router.get("/:id")` 다. `"/posts"` 나 `"/posts/:id"` 가 아니다. 이 라우터가 메인 앱 파일에서 `/posts` 에 마운트되기 때문에, 여기서의 `"/"` 는 실제로 `/posts` 가 되고 `"/:id"` 는 `/posts/:id` 가 된다. 앞부분 `/posts` 는 마운트할 때 한 번만 정하고, 파일 안에서는 그 뒤만 적는다.
- `module.exports = router` — 완성한 라우터를 내보낸다. 이 한 줄이 있어야 메인 앱 파일이 `require` 로 가져갈 수 있다.

댓글 라우터도 모양이 같다.

```
// 댓글 라우터 - 메인 앱 파일에서 "/comments" 경로에 마운트된다.
const express = require("express");
const router = express.Router();

const comments = [
  { id: 1, postId: 1, content: "좋은 글이네요!" },
  { id: 2, postId: 1, content: "감사합니다." },
];

router.get("/", (req, res) => {
  res.json(comments);
});

module.exports = router;
```

게시물과 한 글자도 안 겹친다. 댓글 데이터도, 댓글 라우트도 이 파일 안에만 있다. 게시물 코드를 고쳐도 이 파일은 건드릴 일이 없다.

메인 앱 파일 — 라우터를 모아 붙인다

```
const express = require("express");
const app = express();

app.use(express.json());

// 각 자원의 라우터를 가져와 경로에 마운트한다.
const postsRouter = require("./routes/posts");
const commentsRouter = require("./routes/comments");

app.use("/posts", postsRouter); // /posts 로 시작하는 요청 → 게시물 라우터
app.use("/comments", commentsRouter); // /comments 로 시작하는 요청 → 댓글 라우터

app.listen(3000, () => {
  console.log("http://localhost:3000 에서 실행 중");
});
```

메인 앱 파일에 개별 라우트는 한 줄도 없다. 라우터를 가져와 붙이는 일만 한다.

- `require("./routes/posts")` — 게시물 라우터 파일을 불러온다. 그 파일이 `module.exports = router` 로 내보낸 `router` 가 `postsRouter` 에 담긴다. 경로 앞의 `./` 는 "이 파일과 같은 폴더 기준"이라는 뜻이다.
- `app.use("/posts", postsRouter)` — 경로를 지정한 `app.use` 다. `/posts` 로 시작하는 모든 요청을 `postsRouter` 에게 통째로 넘긴다. 이 "경로에 라우터를 붙이는" 동작을 **마운트(mount)** 라고 한다.

여기서 04장에서 배운 미들웨어가 다시 보인다. `app.use(express.json())` 은 경로 없이 모든 요청에 거는 미들웨어였다. `app.use("/posts", postsRouter)` 는 **경로를 붙인 미들웨어**다. `/posts` 로 시작하는 요청에만 라우터 묶음을 적용한다. 라우터도 결국 미들웨어의 한 형태다.

마운트가 경로를 어떻게 합치는지 보면 전체 그림이 잡힌다.

```

GET /posts/1
  |
app.use("/posts", postsRouter)  ← /posts 로 시작? → postsRouter에 넘김
  |                                앞의 /posts 를 떼고 나머지(/1)를 라우터에 전달
  ▼
router.get("/:id")              ← /1 이 /:id 에 걸림 → req.params.id = "1"

```

요청	마운트	라우터 안 경로	실행되는 핸들러
GET /posts	/posts	/	게시물 라우터의 목록 조회
GET /posts/1	/posts	/:id	게시물 라우터의 단건 조회
GET /comments	/comments	/	댓글 라우터의 목록 조회

파일이 곧 자원 — 이 구조에서는 "어디를 고쳐야 하나"가 분명해진다. 게시물 기능에 손덜 일이 생기면 게시물 라우터 파일만 열면 된다. 댓글은 댓글 라우터 파일, 사용자가 생기면 사용자 라우터 파일을 새로 만들어 메인 앱 파일에 마운트 한 줄을 추가한다. 앱이 자라도 메인 앱 파일은 마운트 목록으로 짧게 유지된다. 뒤에 배울 NestJS의 모듈 구조도 이 "자원별로 가른다"는 발상을 그대로 잇는다.

입력 검증 — Joi 스키마로 규칙을 선언한다

04장에서 본문을 검사하던 코드를 떠올려 보자.

```

if (!title || !content) {
  return res.status(400).json({ message: "title과 content는 필수입니다." });
}

```

이게 시작이다. 그런데 "제목은 두 글자 이상", "본문은 열 글자 이상", "작성자 필수"처럼 규칙이 늘면 `if` 문이 핸들러 첫 머리를 가득 채운다. 라우트마다 그 검사를 또 적어야 하고, 조건 하나를 빠뜨리면 잘못된 데이터가 그대로 들어온다. **Joi**는 이 손검사를 **규칙 선언**으로 바꾸는 라이브러리다. "무엇을 어떻게 검사할지"를 데이터로 적어두면, 검사는 Joi가 대신한다.

규칙을 선언한다 — Joi 스키마

```
const express = require("express");
const Joi = require("joi");
const app = express();

app.use(express.json());

// 게시물 생성 규칙을 선언적으로 정의한다.
const createPostSchema = Joi.object({
  title: Joi.string().min(2).max(100).required(),
  content: Joi.string().min(10).required(),
  author: Joi.string().required(),
});
```

`createPostSchema` 는 게시물을 만들 때 본문이 지켜야 할 **규칙 명세**다. `if` 문이 아니라, 필드마다 조건을 이어 붙인 선언이다. 읽으면 그대로 규칙이 된다.

- `title: Joi.string().min(2).max(100).required()` — 제목은 문자열이고, 두 글자 이상 백 글자 이하이며, 반드시 있어야 한다. 메서드를 점으로 이어 조건을 쌓는다.
- `content: Joi.string().min(10).required()` — 본문은 문자열, 열 글자 이상, 필수.
- `author: Joi.string().required()` — 작성자는 문자열, 필수. 길이 제한은 없다.

`if (!title)` 같은 명령형 검사와 달리, 스키마는 "이래야 한다"는 **상태를 적는다**. 조건이 추가되면 `.email()`, `.integer()` 처럼 메서드 하나를 더 붙이면 된다. 검사 로직을 직접 짜지 않는다.

검증 미들웨어를 짚어낸다 — 고차함수

```
// 검증 미들웨어를 "만들어 주는" 함수(고차함수) — 스키마마다 규칙이 다르므로,
// 스키마를 받아 그에 맞는 (req, res, next) 미들웨어를 짚어내 돌려준다 → validate(스키마A), validate(스키마B)로 재사용.
function validate(schema) {
  return (req, res, next) => {
    const { error } = schema.validate(req.body);
    if (error) {
      return res.status(400).json({
        message: "입력값이 올바르지 않습니다.",
        detail: error.details[0].message,
      });
    }
    next(); // 통과하면 다음(실제 라우트)으로 넘어간다.
  };
}
```

이 장에서 새로 나오는 개념이 `validate` 다. 04장의 미들웨어는 `(req, res, next) => { ... }` 함수 그 자체였다. 그런데 `validate` 는 미들웨어가 아니라, **미들웨어를 만들어 돌려주는 함수**다. 함수를 반환하는 함수를 **고차함수**라고 부른다.

왜 한 겹 더 감쌌나. 검증 미들웨어는 `(req, res, next)` 라는 정해진 모양이라 인자를 더 받을 수 없다. 그런데 검증할 스키마는 라우트마다 다르다 — 게시물 생성은 `createPostSchema`, 수정은 또 다른 스키마. 그래서 바깥 함수 `validate(schema)` 로 스키마를 먼저 받아 기억해두고, 그 스키마를 쓰는 미들웨어를 짚어내 돌려준다.

- `validate(schema)` — 바깥 함수. 스키마를 인자로 받는다.
- `return (req, res, next) => .{.}` — 안에서 진짜 미들웨어를 만들어 돌려준다. 이 미들웨어는 바깥에서 받은 `schema` 를 그대로 쓴다.
- `schema.validate(req.body)` — 본문을 스키마에 비춰 검사한다. 결과에서 `error` 만 구조분해로 꺼낸다. 통과하면 `error` 는 `undefined`, 어기면 무엇이 잘못됐는지 담긴 객체가 들어온다.
- `error.details[0].message` — Joi가 만든 상세 메시지다. `"title" length must be at least 2 characters long` 처럼 어느 필드가 왜 걸렸는지 알려준다. `detail` 로 함께 내려 보내 클라이언트에 원인을 전한다.
- `next()` — 검증을 통과했을 때만 부른다. 부르지 않으면 요청은 라우트에 닿지 못한다. 즉 **통과한 요청만** 실제 핸들러로 넘어간다.

이렇게 만들어 두면 `validate(createPostSchema)`, `validate(updatePostSchema)` 처럼 스키마만 바꿔 끼워 같은 검증 로직을 재사용한다. 검사 코드는 `validate` 안에 한 번만 있고, 규칙은 스키마에 따로 있다.

라우트 앞에 끼운다

```
const posts = [];
let nextId = 1;

// validate(.)를 라우트 앞에 끼우면, 검증을 통과한 요청만 핸들러에 도달한다.
app.post("/posts", validate(createPostSchema), (req, res) => {
  // 04와 같은 방식 - req.body에서 필드를 하나씩 골라 담는다.
  const { title, content, author } = req.body;
  const post = { id: nextId++, title, content, author };
  posts.push(post);
  res.status(201).json(post);
});
```

`app.post` 의 인자가 04장보다 하나 늘었다. 경로 `"/posts"` 와 핸들러 사이에 `validate(createPostSchema)` 가 끼어 있다. Express는 한 라우트에 미들웨어를 여러 개 줄지어 등록할 수 있고, **왼쪽부터 차례로** 실행한다.

- `validate(createPostSchema)` — 이 줄은 실행 시점에 **미들웨어를 만들어** 그 위치에 끼운다. `createPostSchema` 를 검사하는 미들웨어다.
- 검증을 통과하면 그 미들웨어가 `next()` 를 불러 다음, 즉 실제 핸들러로 넘긴다.
- 어기면 그 자리에서 400을 돌려주고 끝난다. **핸들러는 아예 실행되지 않는다.**

그래서 핸들러 안쪽이 깨끗해졌다. 04장에서는 핸들러 첫머리에 `if (!title || !content)` 가 있었지만, 여기서 그 검사가 사라졌다. 핸들러에 도달한 시점에 본문은 이미 규칙을 통과했다고 믿을 수 있기 때문이다. **검사는 미들웨어가, 처리는 핸들러가** 맡는 분업이다.

요청 하나가 거치는 칸을 그리면 이렇다.

POST /posts → express.json() → validate(createPostSchema) — 통과 → next() → 핸들러 → 201

└─ 위반 → 400 (핸들러 미실행)

본문	결과	상태코드
<code>{"title": "제목입니다", "content": "본문은 열 글자 이상", "author": "홍길동"}</code>	통과 → 생성	201
<code>{}</code>	<code>title</code> 필수 위반	400
<code>{"title": "a", "content": "!.", "author": "홍길동"}</code>	<code>title</code> 두 글자 미만	400

선언이 검사보다 강하다 — `if` 검사는 빠뜨리면 그냥 통과한다. 스키마는 적어두지 않은 필드의 조건이 빠질 뿐, 적은 규칙은 빠짐없이 검사된다. 규칙이 한곳에 모여 있어 "이 자원의 입력 규칙이 뭐였더라"를 스키마만 보면 안다. NestJS에서는 이 발상이 클래스와 데코레이터(`class-validator`)로 한 번 더 다듬어져 나온다.

정리 표 — 04장과 무엇이 달라졌나

주제	04장 (입문)	05장 (중급)
실패 응답	라우트마다 <code>res.status(.).json(.)</code>	<code>throw</code> → 에러 핸들러 한곳
없는 경로	처리 없음(기본 404)	마지막 <code>app.use</code> 404 미들웨어
라우트 위치	한 파일에 전부	자원별 파일 + 메인 앱 파일 마운트
입력 검증	핸들러 안 <code>if</code> 손검사	Joi 스키마 + 검증 미들웨어

흔한 실수

- 에러 핸들러의 인자를 셋으로 줄인다. → `(err, req, res)` 로 쓰면 Express가 일반 미들웨어로 본다. `next` 를 안 쓰더라도 인자 네 개를 다 적어야 에러 핸들러로 인식된다.
- 404 미들웨어를 라우트 위에 둔다. → 모든 요청을 가로채 전부 404가 나간다. 404 핸들러와 에러 핸들러는 반드시 맨 아래다.

- 라우터 파일에 `/posts` 를 또 적는다. → 메인 앱 파일에서 `/posts` 에 마운트했는데 라우터 안에서도 `router.get("/posts")` 로 적으면 실제 경로가 `/posts/posts` 가 된다. 마운트 뒤 경로만 적는다.
- `module.exports = router` 를 빠뜨린다. → 메인 앱 파일의 `require` 가 빈 객체를 가져와 마운트가 안 된다.
- 검증 미들웨어에서 통과 시 `next()` 를 안 부른다. → 정상 요청도 라우트에 닿지 못하고 멈춘다.
- `validate(createPostSchema)` 대신 `validate` 만 넣는다. → 미들웨어가 아니라 고차함수 자체를 등록하게 돼 동작하지 않는다. 호출해서 반환된 미들웨어를 끼워야 한다.

직접 해보기

1. 에러 처리에서 없는 글(`!post`)도 `throw` 로 바꿔 보자. `err.status = 404` 를 붙여 던지고, `res.status(404).` 줄을 지운다. 에러 핸들러 하나로 400과 404가 모두 처리되는지 `curl` 로 확인한다.
2. 라우터 분리 구조에 사용자 라우터 파일을 새로 만들어 보자. 사용자 목록을 돌려주는 `router.get("/")` 만 두고, 메인 앱 파일에 `app.use("/users", ...)` 한 줄을 추가해 `GET /users` 가 동작하게 한다.
3. 입력 검증에 수정용 스키마 `updatePostSchema` 를 만들어 보자. 모든 필드를 선택(`required()` 제거)으로 두고, `app.put("/posts/:id", validate(updatePostSchema), ...)` 에 끼워 `validate` 가 정말 재사용되는지 확인한다.

정리

- 라우트마다 적던 실패 응답은 `throw` 로 던지고 인자 4개 에러 핸들러 하나가 받는다. 없는 경로는 맨 아래 `app.use 404` 미들웨어가 잡는다.
- `express.Router()` 로 라우트를 자원별 파일로 가르고, 메인 앱 파일은 `app.use("/경로", 라우터)` 로 마운트만 한다. 라우터 파일 안에서는 마운트 뒤 경로만 적는다.
- `Joi` 스키마로 입력 규칙을 선언하고, 고차함수 `validate(schema)` 로 검증 미들웨어를 찍어내 라우트 앞에 끼운다. 통과한 요청만 핸들러에 닿는다.
- 세 도구 모두 04장의 미들웨어 한 개념을 다른 자리에 쓴 것이다.

다음 장(06·날짜리 + 게시판)에서는 서버를 끄면 사라지던 인메모리 배열을 버리고, SQLite에 SQL로 직접 데이터를 저장한다.